

ShopEasy Term Project: Reflection Report

SE3004 Software Testing · Mahmut Yusuf Öz (220717004)

Bugs each technique revealed

The five techniques produced very different kinds of feedback. Specification-based testing (Task 1) on `PriceCalculator` did not surface a defect in the formula itself; the implementation already handled the partitions I picked, including the 0% and 100% boundary points for both discount and tax. What it did expose was how easy it is to skip the boundaries by accident. The boundary-matrix parameterized test forced me to write the four extreme combinations that I would otherwise have left out, and the off-point invalid-input tests showed that without the asserts added in Task 3 the class would silently compute negative totals for a discount rate above 100%. Structural testing (Task 2) drove `ShoppingCart` branch coverage to 100% (0 of 17 branches missed) and did not uncover a bug either, but the optional PIT mutation run was more interesting: out of 65 generated mutations only 42 were killed (65% mutation score). The most informative survivor was the `ConditionalsBoundary` mutation on `ShoppingCart.updateQuantity` that rewrites the guard `quantity <= 0` to `quantity < 0`. My test for `quantity == 0` still passed against the mutant because the downstream `CartItem.setQuantity` has its own identical guard that throws `IllegalArgumentException`, so the violation surfaced anyway through a different code path. A second surviving mutant, on `CartItem.setQuantity` itself, had the same shape: no test exercised `setQuantity(0)` directly. Both could be killed by asserting on the exception message text (so the source of the throw is visible) or by writing a unit test against `CartItem` in isolation.

Design-by-Contract (Task 3) was the technique that actually changed the production code's behaviour. Before I added the asserts, `ShoppingCart.applyDiscount` happily accepted a rate of 150% and returned a negative total, and `addItem(null, 1)` would only fail much later inside the for-loop with a `NullPointerException`. After adding the pre- and post-condition asserts, both inputs are rejected immediately and with a descriptive message. The contract tests then turn those informal expectations into something the build enforces. Property-based testing (Task 4) generated $4 \times 1000 = 4000$ cases across four properties (identity, monotonicity over discount, boundedness, cart commutativity) and all of them held. Monotonicity in particular felt like a strong specification because a single sign-flip bug in the formula would have been caught in a single iteration. Mocks (Task 5) confirmed the orchestration rules of `OrderProcessor`: `charge()` is never invoked when even one item fails the inventory check, and `isAvailable()` is called once per cart line.

Most effective technique per unit of effort

Honestly, Design-by-Contract felt like the best return for the time invested. Each assert is a one-line addition to the production class, but once the code is compiled with `-ea` every single test in the rest of the project, including the 4000 jqwik samples, is implicitly checking those contracts on every invocation. The two clearest defects I noticed in the whole project (out-of-range discount, null product) were both caught by contracts written in under a minute each.

Property-based testing was a close second, mostly because of how cheaply jqwik produced thousands of inputs from a few `@DoubleRange` annotations and one custom `@Provide` method. The catch is that PBT only works well when you can phrase the invariant cleanly; for one-off business rules, a parameterized JUnit test is usually faster to write.

The trade-off of mocking (Task 5)

Mocking is what made Task 5 possible at all. Without test doubles I would have had to stand up a real inventory store and a real payment processor just to verify that `OrderProcessor.process` aborts on a failed inventory check. With Mockito I was able to inject deterministic returns from `isAvailable()` and `charge()` and then call `verify(paymentGateway, never()).charge(...)` to prove that the abort path actually skips the payment step, which a real gateway would never let me observe cleanly. The cost is that mocked tests can only assert on the contract I imagine the collaborator has. If the real `InventoryService` ever changed its return type or its idea of "available" (for example, returning a quantity instead of a boolean), my tests would still pass against the mock while production would break. Mocks also do not catch coordination bugs, like decrementing inventory twice or charging on a partially fulfilled cart; those need integration tests against a real or in-memory backend.

Where the suite sits on the testing pyramid

All 78 tests in this project are unit tests in Aniche's sense: each one targets a single class, runs in well under a second, and never touches the network, the file system, or another process. They sit at the bottom of the pyramid, exactly where they belong. What the pyramid asks for that is not here is the next layer up, integration tests that wire two or more real components together. For example, there is no test that builds a `ShoppingCart`, feeds it into a `PriceCalculator`, and then routes the result through a real in-memory `InventoryService`; `OrderProcessor` is only exercised against Mockito mocks. The top of the pyramid (end-to-end tests against an actual HTTP endpoint or UI) is missing too, but that is fair because ShopEasy ships as a library and has no outer entry point yet. The mutation score (65%) is another reminder that even with 100% branch coverage there are observable behaviours the unit tests do not pin down.

If I had one more week

I would spend it on two things. First, integration tests: write a small in-memory `InventoryService` implementation backed by a `Map<Product, Integer>` and a stateful fake `PaymentGateway` that tracks per-customer balances, then exercise `OrderProcessor` end-to-end against them. Those tests would catch coordination bugs that mocks cannot, like double-decrement or wrong total after a partial cart update. Second, I would attack the remaining surviving mutants from PIT, especially the `NO_COVERAGE` ones in `OrderProcessor` and `Order`, and try to push the mutation score from 65% toward 90%. A side goal would be a small concurrency test for `ShoppingCart.addItem` under multiple threads adding the same product, since the for-loop and the `items.add` call together are not thread-safe and could lose updates under contention.